

Guia Prático de Git e GitHub

Controle de Versão para Projetos de Software

Material de apoio didático

Prof. Wagner Wesley Leme Ramalho

Curso de Desenvolvimento de Software Multiplataforma

1. Introdução ao Controle de Versão

Controle de versão é um sistema que permite registrar alterações feitas em arquivos ao longo do tempo. Ele possibilita recuperar versões anteriores do projeto, acompanhar o histórico de mudanças e facilitar o trabalho em equipe. Em projetos de software, o controle de versão é essencial para organizar o desenvolvimento e evitar perda de código.

Este guia foi elaborado como material de consulta para utilização de Git e GitHub nas atividades da disciplina. Durante as aulas os comandos foram apresentados e praticados. Utilize este documento sempre que precisar relembrar o fluxo de trabalho utilizado no desenvolvimento de software em equipe.

2. O que é Git

Git é um sistema de controle de versão distribuído amplamente utilizado no desenvolvimento de software. Ele permite registrar alterações no código, criar ramificações de desenvolvimento e colaborar com outras pessoas de forma organizada.

3. O que é GitHub

GitHub é uma plataforma online que hospeda repositórios Git. Ela permite compartilhar código, colaborar com outras pessoas, gerenciar versões de projetos e acompanhar contribuições. Com o GitHub, é possível armazenar o código em um repositório remoto e trabalhar em equipe de forma eficiente.

4. Conceitos Fundamentais

Repositório: Local onde os arquivos do projeto são armazenados e gerenciados pelo Git.

Commit: Registro das alterações feitas no projeto em um determinado momento.

Branch: Linha de desenvolvimento independente usada para criar novas funcionalidades ou corrigir problemas.

Merge: Processo de combinar alterações de diferentes branches.

5. Instalação do Git

1. Acesse o site: <https://git-scm.com>
2. Baixe a versão adequada para o seu sistema operacional.
3. Instale utilizando as opções padrão do instalador.
4. Verifique a instalação executando no terminal: `git --version`

6. Configuração Inicial

Configure seu nome e e-mail para identificar seus commits:

```
git config --global user.name "Seu Nome"
```

```
git config --global user.email "seu@email.com"
```

Para verificar as configurações:

```
git config --global --list
```

7. Comandos Básicos do Git

git init: Inicializa um repositório Git em um diretório.

git clone: Copia um repositório remoto para sua máquina.

git add: Adiciona arquivos à área de preparação (stage).

git commit: Registra as alterações no histórico do projeto.

git status: Mostra o estado atual do repositório.

git log: Exibe o histórico de commits.

git branch: Lista ou cria branches.

git checkout: Muda para outra branch.

git merge: Combina alterações de diferentes branches.

8. Clonar um Repositório (Fluxo Recomendado)

Clonar o repositório remoto:

```
git clone Link_do_Repositório
```

Acessar o repositório clonado:

```
cd nome_do_repositório
```

Abrir no editor VSCode

```
code .
```

9. Iniciando uma Nova Tarefa

Verificar se o repositório local está atualizado

```
git pull origin main
```

Criar uma nova branch para a tarefa e já mudar para ela:

```
git checkout -b nome_da_branch
```

Enviar a nova branch para o repositório remoto:

```
git push origin nome_da_branch
```

Observação: Ao finalizar uma tarefa, ou modificação, fazer commit frequentes e atômicos, para ficar fácil de reverter caso seja necessário.

Após implementar as alterações, execute os comandos definidos no item 11 (Final do Dia / Término da Tarefa).

10. Começo do Dia

Explicativo: Esse processo deve ser executado sempre que você iniciar o trabalho no projeto. Pois deve-se verificar se alguém da equipe subiu alguma modificação na Branch de referência (main) do remoto, porque sempre que o remoto tem uma atualização que ainda não está no seu ambiente local, é necessário fazer o rebase.

Entendendo melhor o Rebase e quando devemos utilizar.

Quando está na Branch de referência e roda o comando git pull, ele irá validar se possui modificações que ainda não está no seu ambiente. Quando existe ele baixa essas atualizações e informa no terminal que foi atualizado. Isso faz com que as Branchs criadas a partir da sua referência, para as tarefas na qual está trabalhando ainda, estejam desatualizadas, porque considera a versão anterior ainda onde não possui essa atualização. Para trazer as atualizações baixadas na Main para a Branch que estamos trabalhando em uma tarefa é necessário utilizar o comando Rebase. Porém precisamos entender como o rebase atua sobre as nossas branches. Ao rodar o comando rebase, ele pega todos os commits que possui na Main após a versão que você utilizou para criar sua Branch no início da tarefa.

Exemplo: Quando criei minha Branch a partir da Main para trabalhar em uma tarefa a hash do commit da Main era: 2c1ee39. Quando alguém da equipe subiu atualizações (Fez o merge na Main) gerou um novo commit com a Hash: c7cf513. Isso faz com que minha Branch da tarefa não tenha essa modificação, e o git considera que sua Branch está desatualizada pois está com commits atras.

Por isso devemos utilizar o rebase, para que o git pegue tudo que está na hash c7cf513 e

adicione na sua Branch da tarefa.

Ai que entra um ponto de atenção: Para que o git consiga trazer oque está na hash nova que ainda não tem na sua Branch ele rescreve o histórico da sua Branch o que faz com que ela fique diferente da versão enviada anteriormente ao repositório remoto na Branch da tarefa. Ou seja, se fizer o rebase e tentar fazer um push para sua Branch remota, vai conflitar. Então sempre que fizer o rebase é necessário informar no próximo commit. Para isso adicionamos a flag “**--force-with-lease**”.

Abaixo descrevo duas rotinas, sendo a que não ocorre o processo de rebase, que é utilizada quando roda o comando pull e não tinha atualizações e outra quando existe atualizações e precisa levar para a branch que estava trabalhando.

Rotina onde não tem atualizações no remoto:

Acessar a Branch Main:

git checkout main

Verificar se existe atualizações e caso exista baixar:

git pull origin main

Verificar a mensagem no terminal. Se não houver atualizações, mude para a branch da tarefa e trabalhe normalmente:

git checkout Branch_tarefa

Rotina onde possui atualizações no remoto:

git checkout main

git pull origin main

O terminal irá exibir as atualizações baixadas.

Próximo passo fazer o rebase seguindo os comandos definidos no item 12 (Atualizando sua Branch com master (Rebase Seguro)) deste arquivo.

11. Final do Dia / Término da Tarefa

Verificar os arquivos alterados:

git status

Adicionar os arquivos alterados na área stage.

Para adicionar todos os arquivos alterados usar o comando:

git add .

Para adicionar o arquivo pelo nome usar o comando:

```
git add nome_do_arquivo
```

Fazer o commit para registrar a alteração:

```
git commit -m "Mensagens descritivas e concisas"
```

Atualizar o remoto com as novas mudanças

```
git push origin nome_da_branch
```

12. Atualizando sua Branch com main (Rebase Seguro)

Após verificar a mensagem informando que teve atualizações é necessário mudar para a branch da tarefa onde precisa atualizar:

```
git checkout nome_branch_tarefa
```

Rodar o comando que vai trazer a atualização baixada na Main:

```
git rebase main
```

Ao rodar o rebase, pode ocorrer sucesso ou conflito caso a versão atualizada na master tenha alguma linha modificada na qual você também tenha modificado na sua branch.

Se houver conflitos:

Resolva manualmente acessando o arquivo e mantendo o código que deve ficar.

Após isso rodar os comandos:

```
git add nome_do_arquivo
```

```
git rebase --continue
```

Feito isso o código da atualização da main vai estar também na sua Branch.

Próximo passo testar se não quebrou as modificações feita na sua Branch.

Confirmando que está tudo certo é necessário subir a nova versão no repositório remoto. Lembrando que o rebase rescreveu seu histórico na Branch da tarefa então para fazer o envio ao repositório remoto é necessário utilizar o comando:

```
git push --force-with-lease
```

Feito isso, já pode voltar a trabalhar normalmente em sua Branch.

13. Merge

Quando precisa levar para a Branch de referência o código que desenvolveu na branch da tarefa é necessário seguir os seguintes passos:

Mudar para a branch de referência Main:

```
git checkout main
```

Conferir se tem atualizações no repositório remoto:

```
git pull origin main
```

#Se tiver atualizações, rodar o fluxo do **rebase (item 12)** antes de fazer o merge para adicionar as modificações da branch de referência na branch da tarefa. Feito isso deve repetir o processo descrito neste merge.

Se não tiver atualizações, seguir com os comandos abaixo.

Aplicar as mudanças que estão na Branch de tarefa na main:

```
git merge nome_da_branch_de_tarefa
```

É importante observar que o comando `git merge` pode resultar em conflitos. Se ocorrerem conflitos durante o merge, você precisa resolvê-los manualmente:

Edite os arquivos que contêm os conflitos.

Marque os conflitos como resolvidos.

Adicione os arquivos resolvidos ao índice:

```
git add nome_do_arquivo
```

Faça o commit das alterações resolvidas:

```
git commit -m "Mensagem de commit"
```

Após resolver os conflitos e fazer o commit, você pode continuar o merge com:

```
git merge --continue
```

Se desejar cancelar o merge e voltar ao estado anterior, use o comando:

```
git merge --abort
```

14. Trabalhando com Branches

Garantir que está na Branch de referência:

```
git checkout main
```

Garantir que a Branch está atualizada:

```
git pull origin main
```

Criar uma nova Branch:

```
git checkout -b nome-da-branch
```

Enviar a branch para o repositório remoto:

```
git push origin nome-da-branch
```

15. Manutenção da Branch

Sempre que terminar a tarefa e fizer o merge na main é necessário fazer a manutenção nas Branch. Ou seja, excluir a Branch remota e local. Para isso deve garantir e rodar a main e confirmar que toda a implementação feita na Branch da tarefa está funcionando corretamente na main.

Excluir a branch remota:

```
git push origin :nome_da_branch_que_desejo_excluir
```

Importante: ao excluir uma branch remota, lembre-se de que há um espaço após o nome do remoto e os dois pontos devem estar colados ao nome da branch, sem espaços. Essa ação é irreversível, então certifique-se bem de que não precisará mais da branch antes de excluí-la.

Excluir a branch local:

```
git branch -D nome_da_branch_que_desejo_excluir
```

Observações: Para excluir a branch local, o -D é utilizado para forçar a exclusão, então tenha certeza de que deseja excluir a branch localmente. Se precisar baixar novamente a branch remota após excluí-la localmente, use git fetch origin e git checkout nome_da_branch_no_remoto.

16. Resolução de Conflitos

Durante um merge podem ocorrer conflitos quando duas pessoas modificam a mesma parte de um arquivo. Nesse caso, é necessário editar manualmente o arquivo, resolver o conflito e depois adicionar e confirmar as alterações novamente.

17. Boas Práticas Profissionais

Trabalhar sempre em branches separadas.

Evitar alterações diretas na branch principal (main).

Escrever mensagens de commit claras e objetivas.

Atualizar sua branch frequentemente para evitar conflitos.

Utilizar “.gitignore” para ignorar arquivos desnecessários.